

Python Practice Pack: Arrays (Lists)

Concept Guide + Common Programs + Practice Questions

- [Part 1: Core Concepts — Array \(List\) Syntax & Logic](#)
- [Part 2: Important Array \(List\) Functions — Syntax & Examples](#)
- [Part 3: Common Array Programs \(with Line-by-Line Explanation\)](#)
- [Part 4: Practice Questions \(Medium → Hard\)](#)

Part 1: Core Concepts — Array (List) Syntax & Logic

In Python, the built-in **list** is the everyday “array.” (Python also has a stricter `array` module and NumPy arrays for numeric-only, fixed-type data — mentioned briefly at the end of this section.)

1.1 Creating and Accessing Arrays

```
arr = [10, 20, 30, 40, 50]      # creation
arr = list(range(1, 6))        # [1, 2, 3, 4, 5]
arr = [0] * 5                  # [0, 0, 0, 0, 0] – repeated value

arr[0]      # 10   (first element, index starts at 0)
arr[-1]     # 50   (last element, negative indexing)
arr[1:4]    # [20, 30, 40] (slice: index 1 up to, NOT including, 4)
arr[::-1]   # [50, 40, 30, 20, 10] (reversed copy)
len(arr)    # 5    (number of elements)
```

Logic: Just like strings, list indices start at `0`, negative indices count from the end, and slicing uses `[start:stop:step]` with `stop` excluded.

1.2 Modifying Arrays

```
arr[0] = 99      # update by index – lists ARE mutable (unlike strings)
arr.append(60)   # add to the end
arr.insert(2, 25) # insert 25 at index 2, shifting others right
arr.remove(25)   # removes the FIRST matching value (by value, not index)
arr.pop()        # removes & returns the LAST element
arr.pop(0)       # removes & returns element at index 0
del arr[1]       # deletes element at index 1 (no return value)
arr.clear()      # empties the whole list
```

Logic: Unlike strings, lists are **mutable** — you can change elements in place without creating a new object. This is the biggest structural difference from string manipulation.

1.3 Traversal Patterns (the “loop logic” of arrays)

Simple traversal

```
for item in arr:
    print(item)
```

Traversal with index (when you need position too)

```

for i in range(len(arr)):
    print(i, arr[i])

# or, more Pythonic:
for i, value in enumerate(arr):
    print(i, value)

```

Two-pointer traversal (common for reversal, palindrome-checks, pair-sums)

```

left, right = 0, len(arr) - 1
while left < right:
    # compare/swap arr[left] and arr[right]
    left += 1
    right -= 1

```

Sliding window traversal (common for subarray sum/max problems)

```

window_sum = sum(arr[:k]) # sum of first k elements
for i in range(k, len(arr)):
    window_sum += arr[i] - arr[i - k] # add new element, remove oldest

```

Logic: Two-pointer and sliding-window are not new syntax — they're **patterns** built from ordinary `while` / `for` loops, chosen because they avoid re-scanning the same elements repeatedly (which nested loops often do wastefully).

1.4 Nested Lists (2D Arrays / Matrices)

```

matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

matrix[1][2] # 6 → row index 1, column index 2

for row in matrix:
    for value in row:
        print(value, end=" ")

```

Logic: A 2D array is simply a list of lists. The outer loop walks through each row; the inner loop walks through each value within that row.

1.5 List Comprehensions (compact loop + conditional)

```

squares = [x**2 for x in range(1, 6)] # [1, 4, 9, 16, 25]
evens = [x for x in range(10) if x % 2 == 0] # loop + filter condition
labels = ["even" if x % 2 == 0 else "odd" for x in range(5)] # loop + if-else

```

Logic: `[expression for item in iterable if condition]` packs a `for` loop (and optional `if` filter, or inline `if-else`) into a single line that builds a new list.

1.6 A Note on `array` module and NumPy

```

from array import array
nums = array('i', [1, 2, 3]) # 'i' = typecode for integers; ALL elements must be same type

import numpy as np
nums = np.array([1, 2, 3]) # supports fast vectorized math: nums * 2 → [2, 4, 6]

```

Logic: Plain Python lists can mix types and resize freely — flexible but slower for heavy numeric work. `array` and

NumPy arrays are stricter (fixed type) but much faster for large-scale numeric operations. For the questions in this pack, plain lists are used throughout, since the focus is on loop/conditional logic.

Part 2: Important Array (List) Functions — Syntax & Examples

Function/Method	Syntax	What it does	Example
<code>len()</code>	<code>len(arr)</code>	Number of elements	<code>len([1,2,3]) → 3</code>
<code>append()</code>	<code>arr.append(x)</code>	Adds <code>x</code> to the end	<code>[1,2].append(3) → [1,2,3]</code>
<code>insert()</code>	<code>arr.insert(i, x)</code>	Inserts <code>x</code> at index <code>i</code>	<code>[1,3].insert(1,2) → [1,2,3]</code>
<code>extend()</code>	<code>arr.extend(other)</code>	Appends all elements of another iterable	<code>[1,2].extend([3,4]) → [1,2,3,4]</code>
<code>remove()</code>	<code>arr.remove(x)</code>	Removes first occurrence of value <code>x</code>	<code>[1,2,2].remove(2) → [1,2]</code>
<code>pop()</code>	<code>arr.pop(i=-1)</code>	Removes & returns element at index <code>i</code> (default: last)	<code>[1,2,3].pop() → 3</code> , list becomes <code>[1,2]</code>
<code>index()</code>	<code>arr.index(x)</code>	Returns index of first occurrence of <code>x</code>	<code>[5,8,2].index(8) → 1</code>
<code>count()</code>	<code>arr.count(x)</code>	Counts occurrences of <code>x</code>	<code>[1,2,2,3].count(2) → 2</code>
<code>sort()</code>	<code>arr.sort(reverse=False, key=None)</code>	Sorts the list in place	<code>[3,1,2].sort() → [1,2,3]</code>
<code>sorted()</code>	<code>sorted(arr)</code>	Returns a new sorted list (original unchanged)	<code>sorted([3,1,2]) → [1,2,3]</code>
<code>reverse()</code>	<code>arr.reverse()</code>	Reverses the list in place	<code>[1,2,3].reverse() → [3,2,1]</code>
<code>reversed()</code>	<code>list(reversed(arr))</code>	Returns a reverse iterator (wrap in <code>list()</code> to see it)	<code>list(reversed([1,2,3])) → [3,2,1]</code>
<code>min()</code> / <code>max()</code>	<code>min(arr)</code> / <code>max(arr)</code>	Smallest / largest element	<code>max([4,1,9]) → 9</code>
<code>sum()</code>	<code>sum(arr)</code>	Total of all elements	<code>sum([1,2,3]) → 6</code>
<code>enumerate()</code>	<code>enumerate(arr)</code>	Pairs each element with its index	<code>list(enumerate(['a','b'])) → [(0,'a'),(1,'b')]</code>
<code>zip()</code>	<code>zip(arr1, arr2)</code>	Pairs elements from two (or more) lists positionally	<code>list(zip([1,2],['a','b'])) → [(1,'a'),(2,'b')]</code>
<code>list()</code> / copy	<code>arr2 = arr.copy()</code>	Creates a shallow copy (avoids aliasing bugs)	<code>b = a.copy()</code>
slicing assignment	<code>arr[1:3] = [9,9]</code>	Replaces a slice with new values	<code>[1,2,3,4][1:3]=[9,9] → [1,9,9,4]</code>
<code>any()</code> / <code>all()</code>	<code>any(arr)</code> / <code>all(arr)</code>	True if <i>any/all</i> elements are truthy	<code>any([0,0,1]) → True</code>
<code>map()</code>	<code>list(map(func, arr))</code>	Applies a function to every element	<code>list(map(str, [1,2])) → ['1','2']</code>
<code>filter()</code>	<code>list(filter(func, arr))</code>	Keeps only elements where <code>func</code> returns True	<code>list(filter(lambda x: x>2, [1,2,3])) → [3]</code>

Part 3: Common Array Programs (with Line-by-Line

Explanation)

These are frequently-asked “classic” array problems. Each is shown once with a clean solution and a **line-by-line walkthrough**, so you can use them as worked examples before moving to the practice questions.

Program 1: Find the Largest and Smallest Element

```
arr = [12, 45, 2, 41, 31, 10, 8]

largest = arr[0]
smallest = arr[0]

for num in arr[1:]:
    if num > largest:
        largest = num
    if num < smallest:
        smallest = num

print("Largest:", largest)
print("Smallest:", smallest)
```

Line-by-line: 1. `arr = [...]` — the input list we’re scanning. 2. `largest = arr[0]` — start by *assuming* the first element is the largest (a safe starting guess). 3. `smallest = arr[0]` — same idea, assume the first element is also the smallest for now. 4. `for num in arr[1:]` — loop through the **rest** of the list (skipping index 0, since it’s already our starting guess). 5. `if num > largest` — if the current number beats our current largest guess... 6. `largest = num` — ...update the guess. 7. `if num < smallest` — separately (not `elif`, since a number can’t be both greater than largest AND less than smallest) check against the smallest guess. 8. `smallest = num` — update if it’s a new smallest. 9-10. Print both final results after the loop completes.

Program 2: Reverse an Array In-Place (Two-Pointer)

```
arr = [10, 20, 30, 40, 50]

left = 0
right = len(arr) - 1

while left < right:
    arr[left], arr[right] = arr[right], arr[left]
    left += 1
    right -= 1

print("Reversed:", arr)
```

Line-by-line: 1. `arr = [...]` — the list to reverse. 2. `left = 0` — pointer starting at the beginning. 3. `right = len(arr) - 1` — pointer starting at the last valid index. 4. `while left < right` — keep swapping as long as the pointers haven’t met or crossed in the middle. 5. `arr[left], arr[right] = arr[right], arr[left]` — Python’s tuple-swap syntax: swaps the values at the two pointer positions in a single line, without needing a temporary variable. 6. `left += 1` — move the left pointer inward (rightward). 7. `right -= 1` — move the right pointer inward (leftward). 8. Once `left` and `right` meet or cross, the loop stops — the array has been reversed **in place** (no new list was created).

Program 3: Sum and Average of Array Elements

```
arr = [15, 22, 8, 19, 31]

total = 0
for num in arr:
    total += num

average = total / len(arr)
```

```
print("Sum:", total)
print("Average:", average)
```

Line-by-line: 1. `arr = [...]` — the input list. 2. `total = 0` — accumulator variable, initialized before the loop (a very common pattern — always start accumulators at a “neutral” value: `0` for sums, `1` for products). 3. `for num in arr:` — visit every element once. 4. `total += num` — add the current element to the running total (shorthand for `total = total + num`). 5. `average = total / len(arr)` — after the loop, divide the final total by the count of elements. Using `/` (not `//`) ensures a proper decimal average, not an integer-truncated one. 6–7. Print both results.

Program 4: Find the Second Largest Element (Single Pass)

```
arr = [12, 45, 2, 41, 31, 10, 8]

largest = second = float('-inf')

for num in arr:
    if num > largest:
        second = largest
        largest = num
    elif num > second and num != largest:
        second = num

print("Second largest:", second)
```

Line-by-line: 1. `arr = [...]` — input list. 2. `largest = second = float('-inf')` — both start at negative infinity, guaranteeing that the very first real number in the array will beat them (safer than assuming `0` or `arr[0]`, especially if the array could contain negative numbers). 3. `for num in arr:` — one single pass through the list. 4. `if num > largest:` — if we found a new overall largest... 5. `second = largest` — ...the *old* largest gets demoted to second place *before* we overwrite it. 6. `largest = num` — now update largest to the new value. 7. `elif num > second and num != largest:` — otherwise, if this number is not the largest, but it *does* beat the current second-largest (and isn't a duplicate of the largest value)... 8. `second = num` — ...update second place. 9. Print the final second-largest value.

Program 5: Count Frequency of Each Element

```
arr = [1, 2, 2, 3, 3, 3, 4]

frequency = {}
for num in arr:
    if num in frequency:
        frequency[num] += 1
    else:
        frequency[num] = 1

for key, value in frequency.items():
    print(key, ":", value)
```

Line-by-line: 1. `arr = [...]` — input list with repeated values. 2. `frequency = {}` — an empty dictionary to store value → count pairs. 3. `for num in arr:` — visit every element. 4. `if num in frequency:` — check if we've seen this value before (checking dictionary **keys**). 5. `frequency[num] += 1` — if seen before, increment its count. 6. `else:` — if this is the first time we've seen this value... 7. `frequency[num] = 1` — ...create a new entry starting at count `1`. 8. `for key, value in frequency.items():` — after building the full frequency map, loop through each key-value pair to display it. 9. Print each element alongside how many times it appeared.

Program 6: Remove Duplicates While Preserving Order

```
arr = [4, 2, 4, 5, 2, 6, 8, 6]
```

```

seen = set()
result = []

for num in arr:
    if num not in seen:
        result.append(num)
        seen.add(num)

print("Without duplicates:", result)

```

Line-by-line: 1. `arr = [...]` — input list with duplicates. 2. `seen = set()` — a set for fast “have I seen this before?” lookups (sets check membership faster than lists for large data). 3. `result = []` — the final de-duplicated list, built in the original order. 4. `for num in arr:` — visit elements in their original order (important — this is what preserves order, unlike just doing `list(set(arr))`, which would scramble the order). 5. `if num not in seen:` — only proceed if this value hasn’t appeared yet. 6. `result.append(num)` — add it to our answer list. 7. `seen.add(num)` — mark it as seen so future duplicates get skipped. 8. Print the final list — same order as original, but with duplicates removed.

Program 7: Rotate an Array by K Positions (Right Rotation)

```

arr = [1, 2, 3, 4, 5, 6, 7]
k = 3

k = k % len(arr)
rotated = arr[-k:] + arr[:-k]

print("Rotated:", rotated)

```

Line-by-line: 1. `arr = [...]` — the original array. 2. `k = 3` — rotate right by 3 positions. 3. `k = k % len(arr)` — handles the case where `k` is larger than the array length (e.g., rotating a 7-element array by 10 is the same as rotating by $10 \% 7 = 3$). Also protects against `k=0` edge behavior. 4. `arr[-k:]` — the **last** `k` elements of the array (these will move to the front). 5. `arr[:-k]` — **everything except** the last `k` elements (these will move to the back). 6. `arr[-k:] + arr[:-k]` — concatenating these two slices in this order produces the right-rotated array. 7. Print the result.

Program 8: Find the Missing Number (1 to N)

```

arr = [1, 2, 4, 5, 6]
n = len(arr) + 1

expected_sum = n * (n + 1) // 2
actual_sum = sum(arr)

missing = expected_sum - actual_sum
print("Missing number:", missing)

```

Line-by-line: 1. `arr = [...]` — the array with exactly one number missing from the sequence 1 to `n`. 2. `n = len(arr) + 1` — since one number is missing, the “true” range size is one more than the array’s current length. 3. `expected_sum = n * (n + 1) // 2` — the mathematical formula for the sum of 1 to `n` (Gauss’s formula) — what the total *should* be if nothing were missing. 4. `actual_sum = sum(arr)` — what the total *actually* is, given the array we have. 5. `missing = expected_sum - actual_sum` — the difference between what should be there and what is there reveals exactly the missing value. 6. Print the result — no searching loop needed at all, just arithmetic.

Part 4: Practice Questions (Medium → Hard)

Each question lists its **core concept**, **difficulty**, and hints (lighter for medium, more structured for hard).

Q1. Move All Zeros to the End (*Medium*)

Given an array, move all 0s to the end while keeping the relative order of non-zero elements (in-place, no extra array for the final result).

Hint: Use a “write pointer” that tracks where the next non-zero element should go. Loop through the array once, and whenever you find a non-zero, place it at the write pointer’s position and advance the pointer — then fill the remaining positions with zeros afterward.

Q2. Maximum Difference (max after min) (*Medium*)

Find the maximum value of `arr[j] - arr[i]` such that `j > i` (i.e., the “buy low, sell high” problem — the profit must come from selling *after* buying).

Hint: Track the minimum value seen *so far* as you loop through the array once, and at each step compute the potential profit using the current element minus that running minimum — keep the best profit seen.

Q3. Find the Duplicate AND Missing Number (*Medium/Hard*)

An array contains numbers 1 to `n`, but one number is duplicated and another is missing (so the array still has `n` elements). Find both.

Hint: Compare the *expected* sum and sum-of-squares (or use a frequency count) against the *actual* values — the difference between expected and actual sum, combined with a similar check on squares (or simply counting occurrences), lets you solve for both unknowns.

Q4. Maximum Subarray Sum (Kadane’s Algorithm) (*Medium/Hard*)

Find the contiguous subarray with the largest sum.

Hint: Track two values as you loop: the best sum ending *exactly* at the current position, and the best sum found *anywhere so far*. At each step, decide whether to extend the previous subarray or start fresh from the current element — whichever gives a larger running sum.

Q5. Find All Pairs With a Given Sum (*Medium*)

Given an array and a target sum, find all pairs of elements that add up to the target.

Hint: Use a set to track numbers you’ve already visited. For each new number, check whether `target - number` already exists in that set — if so, you’ve found a pair, without needing nested loops.

Q6. Find the Majority Element (*Medium*)

Find the element that appears more than `n/2` times in the array (guaranteed to exist).

Hint: Try Boyer-Moore voting: keep a “candidate” and a “count.” When count hits 0, pick a new candidate. Increment count when you see the candidate again, decrement otherwise. The final candidate is the majority element.

Q7. Merge Two Sorted Arrays (*Medium*)

Merge two already-sorted arrays into a single sorted array, without using `sort()` on the final result.

Hint: Use two pointers, one for each array, comparing the elements they currently point to and appending the smaller one to the result — advance only the pointer whose element was used. Don’t forget to append any leftover elements once one array runs out.

Q8. Intersection of Two Arrays (Medium)

Find the common elements between two arrays (each element should appear once in the result, even if it repeats in the inputs).

Hint: Converting both arrays to sets and using set intersection (`&`) handles duplicates and lookups efficiently in very few lines — but also try writing it with a loop and an `in` check to see the manual version.

Q9. Flatten a Nested List (Medium)

Given a list of lists (possibly with uneven row lengths), flatten it into a single 1D list.

Hint: A nested `for` loop (outer loop over rows, inner loop over elements in each row) naturally “unwraps” one level of nesting — build a new list by appending each inner element as you encounter it.

Q10. Transpose a Matrix (Medium)

Given a 2D matrix, produce its transpose (rows become columns and vice versa).

Hint: The value at `transposed[j][i]` should equal `matrix[i][j]`. Use nested loops where the outer loop picks the column index and the inner loop picks the row index (i.e., swap the usual loop order) to build each new row of the transposed matrix.

Q11. Find the Leaders in an Array (Hard)

An element is a “leader” if it is greater than all elements to its right. Find all leaders (the last element is always a leader).

Hint: Instead of comparing each element to *every* element on its right (which is slow), loop from the **right end** of the array toward the left, keeping track of the maximum seen so far — any element that beats this running maximum is a leader.

Q12. Trapping Rain Water (Hard)

Given an array representing elevation heights, calculate how much water can be trapped between the bars after it rains.

Hint 1: For each position, the water trapped above it depends on the **shorter** of (the tallest bar to its left) and (the tallest bar to its right), minus the height at that position. **Hint 2:** Precompute two arrays first — `left_max[i]` (tallest bar from the start up to `i`) and `right_max[i]` (tallest bar from `i` to the end) — then loop once more to sum up `min(left_max[i], right_max[i]) - height[i]` for each position (only when positive).

Q13. Equilibrium Index (Medium/Hard)

Find an index where the sum of elements to its left equals the sum of elements to its right.

Hint: Compute the total sum of the array first. Then loop through once, maintaining a running “left sum” — at each index, the “right sum” can be derived as `total - left_sum - current_element`, avoiding the need to re-sum the right side every time.

Q14. Sliding Window Maximum (Hard)

Given an array and a window size `k`, find the maximum element in every window of size `k` as it slides across the array.

Hint 1: A brute-force nested loop (recomputing the max for every window from scratch) works but is slow — think about what information can be *reused* between consecutive windows. **Hint 2:** Use a `deque` (double-ended queue) to store *indices* of useful elements in decreasing order of value — remove indices from the back that are smaller than the current element (they can never be the max again), and remove indices from the front once they fall outside the current window.

Q15. Longest Consecutive Sequence (Hard)

Given an unsorted array, find the length of the longest run of consecutive integers (e.g., `[100, 4, 200, 1, 3, 2]` → the sequence `1, 2, 3, 4` has length 4).

Hint: Put all numbers in a set first for $O(1)$ lookups. For each number, only start counting a new sequence if `number - 1` is **not** in the set (meaning it's a true starting point) — then keep checking `number+1`, `number+2`, ... to measure that sequence's length.

Q16. Sort an Array of 0s, 1s, and 2s (Dutch National Flag) (Medium/Hard)

Sort an array containing only `0`, `1`, and `2` in a single pass, without using `sort()`.

Hint: Use three pointers: `low`, `mid`, and `high`. Elements before `low` are all `0`s, between `low` and `mid` are all `1`s, and after `high` are all `2`s — swap elements based on what `arr[mid]` currently holds, and move the appropriate pointers.

Q17. Spiral Traversal of a Matrix (Hard)

Print all elements of a 2D matrix in spiral order (starting from the top-left, moving right, then down, then left, then up, and repeating inward).

Hint 1: Track four boundaries: `top`, `bottom`, `left`, `right`. Traverse the top row left-to-right, then the right column top-to-bottom, then the bottom row right-to-left, then the left column bottom-to-top — after each pass, shrink the corresponding boundary inward. **Hint 2:** Add a check after each of the four directional passes to confirm the boundaries haven't crossed yet (important for non-square matrices, where one dimension might "finish" before the other).

Q18. Subarray With a Given Sum (Medium/Hard)

Given an array of positive integers and a target sum, find a contiguous subarray that adds up exactly to the target.

Hint: Use a sliding window with two pointers. Expand the window (move the right pointer) while the current sum is less than the target; shrink it from the left (move the left pointer) when the sum exceeds the target — this avoids recomputing sums from scratch for every possible subarray.

Q19. Product of Array Except Self (Hard)

Given an array, return a new array where each element is the product of all other elements, **without using division** and ideally in $O(n)$ time.

Hint 1: Build two auxiliary arrays: `left_products[i]` (product of everything to the left of `i`) and `right_products[i]` (product of everything to the right of `i`). **Hint 2:** The final answer at index `i` is simply `left_products[i] * right_products[i]`. Once comfortable, try optimizing to use only one extra array (building the result in-place using the output array itself for the left pass, then multiplying by a running right-product in a second pass).

Q20. Minimum Number of Jumps to Reach the End (Hard)

Given an array where each element represents the maximum jump length from that position, find the minimum

number of jumps needed to reach the last index.

Hint 1: Think of it in terms of “levels” or “ranges reachable” — track the farthest index reachable with the current number of jumps, and the farthest index reachable if you take one more jump from anywhere within the current range. **Hint 2:** Loop through the array once, and whenever your loop index reaches the boundary of the current jump’s range, that means you *must* jump again — increment the jump counter and extend the range to the farthest point discovered so far.

End of question set. Solutions with three alternative approaches per problem are provided in the companion solutions document.